



Paths to support additional numeric types on the Java platform

Subhead goes here on one line

Joseph D. Darcy ([OpenJDKdarcy](#), [@jddarcy](#), [@jddarcy](#))

Java Platform Group, Oracle
JavaOne, March 17, 2026



Talks at JavaOne have a mixture of two kinds of content

Features that have shipped and are ready to use:

- [Java 26: Better Language, Better APIs, Better Runtime \[LRN1412\]](#)
- [Just-In-Time Compilation for Java Performance: Recent and Ongoing Improvements \[LRN1390\]](#)
- [Native and AI Interoperability with JDK 25 and the FFM API \[LRN1354\]](#)
- [Hands on with modern Java! \[HCK1470\]](#)
- [Collections at the Coal Face \[LRN1253\]](#)
- [Coloring Inside the Lines: Using Java Cryptography Safely and Sanely \[LRN1062\]](#)
- ...

Features that are being worked on and may, or may not, ship in the future:

- [The Past, Present, and Future of Null Safety in Java \[LRN1224\]](#)
- [New and Upcoming Java Language Features \[LRN1389\]](#)
- [Beyond Vector API: A Quest for a Lower Level API \[LRN1435\]](#)
- ...



This talk is firmly in the “features being worked on, may, or may not, ship in the future” category.

Outline

Motivation

What is a type?

What is a number?

How can we support many new numeric types on the platform?

Type classes and operator overloading

Float16 Retrospective and Prospective

Current thoughts on modeling numerics

Design considerations

Summary and Q&A

Motivation: uses of additional numeric types

Better support in multiple areas

Traditional scientific and engineering computation

Education and “paving the on-ramp”

Machine Learning/AI

***Platform* support for numeric types**

Design work in-progress

What additional kinds numbers to include?

How to add them? For a given, type *platform* support means using some combination of current and future

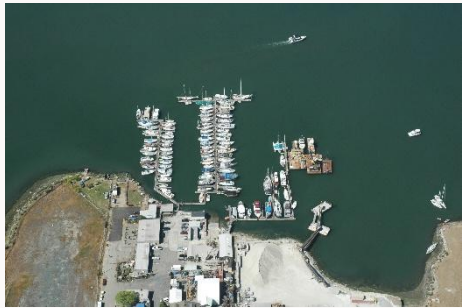
- Java programming language features
- Java virtual machine features
- Core library features

New numeric types may be added to the platform directly, but the intention is *also* to allow 3rd parties to develop and use new numeric types.

In the components above, new features may be aimed at library authors rather than end users.

Essence of the design exercise at the platform level is determining not only *what* to support, but *how* to support it using an allocation of responsibilities among these components.

The following is intended to outline our general product direction. It is intended for information purposes only, and may not be incorporated into any contract. It is not a commitment to deliver any material, code, or functionality, and should not be relied upon in making purchasing decisions. The development, release, timing, and pricing of any features or functionality described for Oracle's products may change and remains at the sole discretion of Oracle Corporation.



Starting Context

Exploring adding new numeric types to the Java platform in anticipation of

- *Value classes* from Project Valhalla ([JEP 401: Value Classes and Objects \(Preview\)](#))
- *Type classes*, a language feature being investigated
- ...

Give feedback on the design of those language features through trial usage.

This talk is a sampling of what we've worked out so far.

- Subject to change in response to future work.
- More explorations expected to follow

Not exhaustive, intend to convey a representative sampling of the considerations at play.

In exploring paths to support additional numerics types:

- What is a *type*?
- What is a *number*?

What is a type?

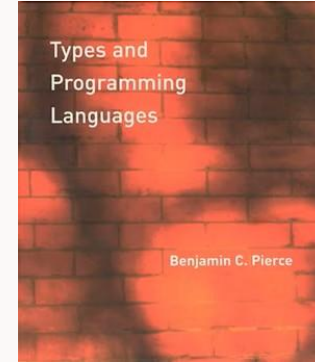


What is a type?

A rich and complicated topic

“A type system is a tractable syntactic method for proving the absence of certain program behaviors by classifying phrases according to the kinds of values they compute.”

[Types and Programming Languages](#), Benjamin C. Pierce



From [Wikipedia](#): “In computer science and computer programming, a data type (or simply type) is a collection or grouping of data values, usually specified by a set of possible values, a set of allowed operations on these values, and/or a representation of these values as machine types.”

Type examples from Java

Historical divide

Reference types

Part of the platform and user-declarable

Sample reference type: `java.lang.String`

- Set of possible values: sequences of characters/code points.
- Can be represented as string literals, "Hello, world."
- Allowable operations
 - String concatenation using "+"
 - Static and instance methods of `java.lang.String`
 - ...
- Objects have *identity*
- Arrays of references
- ...

Primitive types

A fixed set built into the platform; e.g. `int`

- Set of possible values: the integers from -2^{31} to $2^{31} - 1$.
- Can be represented as literals, `-2147483648`, ... `0`, ... `2147483647`
- Allowable operations
 - Arithmetic operations `{+, -, *, /}`
 - Comparison operations `{<, <=, >, >=, ...}`
 - Bit-level and shift operations `{^, ~, <<, ...}`
 - ...
- Primitive values have value, *not* identity
- Dense arrays
- Use in annotations
- ...

OpenJDK Project Valhalla:

“Codes like a class, works like an int.”

Goal: bridge the gap and get the best of both worlds.

See [Growing the Java Language](#) by Brian Goetz, JVMLS 2025

What is a number?



What is a number?

A rich and complicated topic

Mathematicians have had *thousands* of years to think up different kinds of numbers!

Kinds of numbers I encountered in my math and science education before college:

- Positive integers / natural numbers ($\mathbb{N}$)
- Integers ($\mathbb{Z}$)
- Fractions / rational numbers ($\mathbb{Q}$)
- Real numbers ($\mathbb{R}$)
- Complex numbers ($\mathbb{C}$) and imaginary numbers

Many other kinds of numbers used in *mathematics* and *computation*.

Rich *algebraic structures*

Digression on Java Collections and abstract algebra.

An algebraist and a Java developer compare notes on the tools they use.

Collections

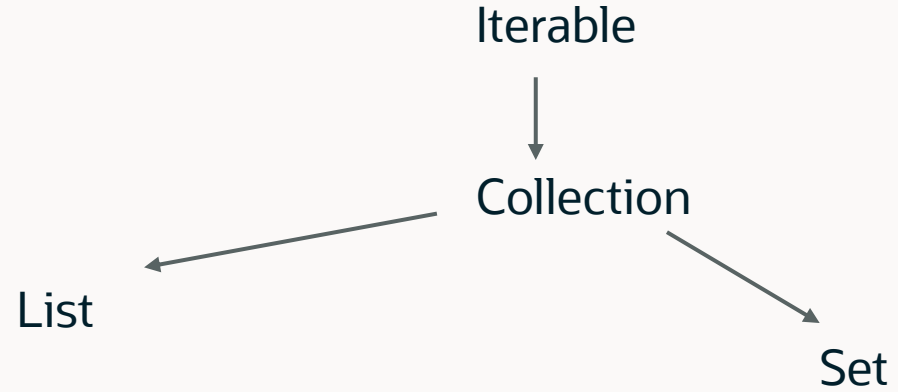
Simplified relations between Collection-related types

Start with basic interfaces, skipping Maps for now



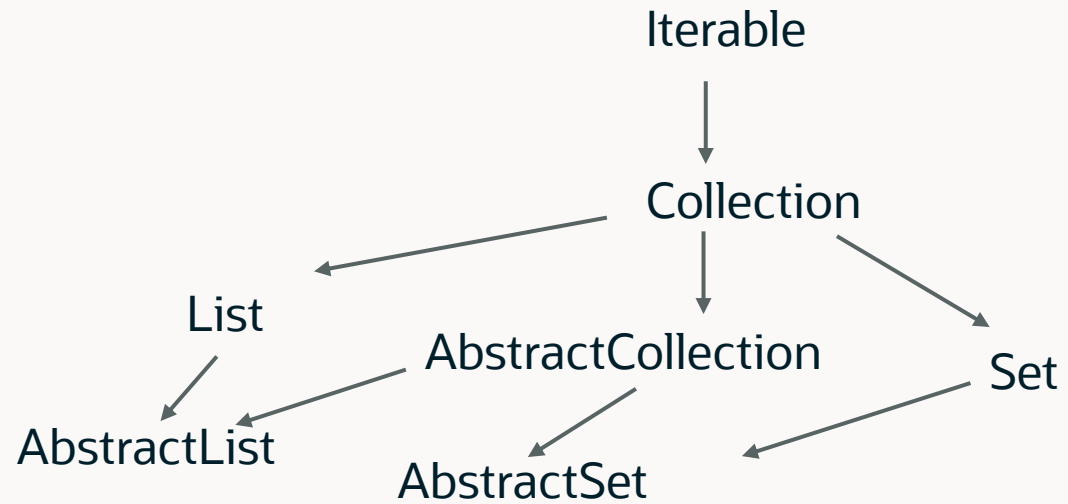
Simplified relations between Collection-related types

Iterable superinterface



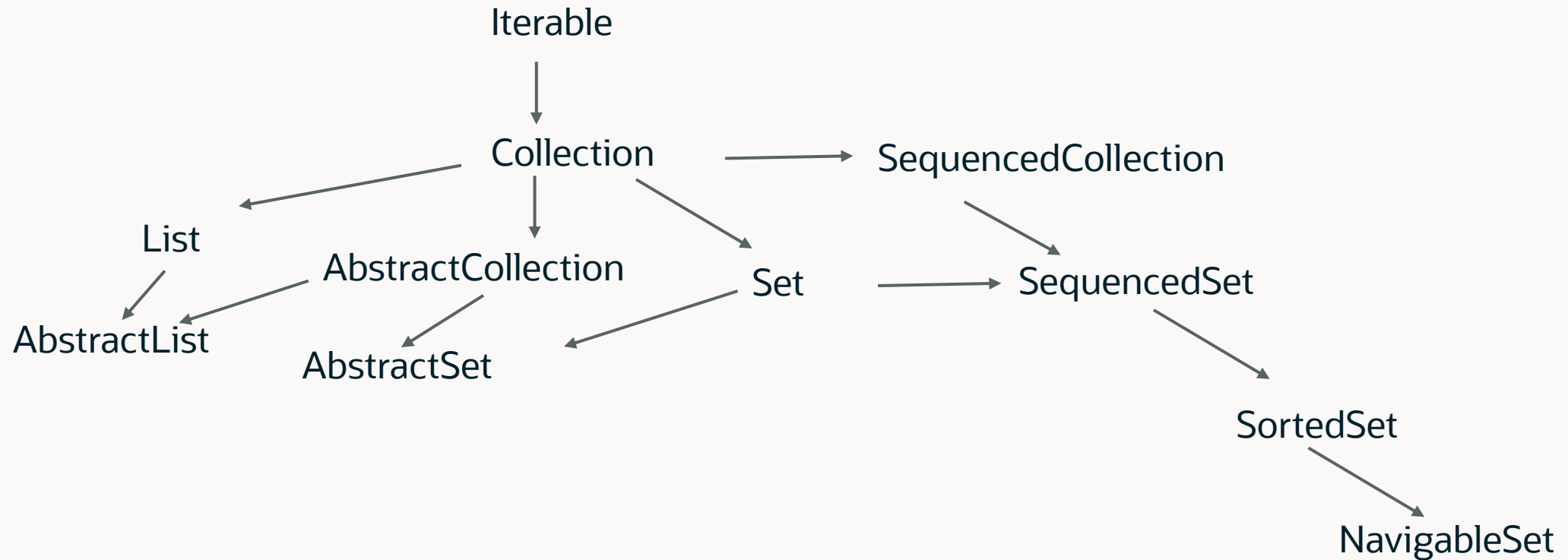
Simplified relations between Collection-related types

Add abstract classes



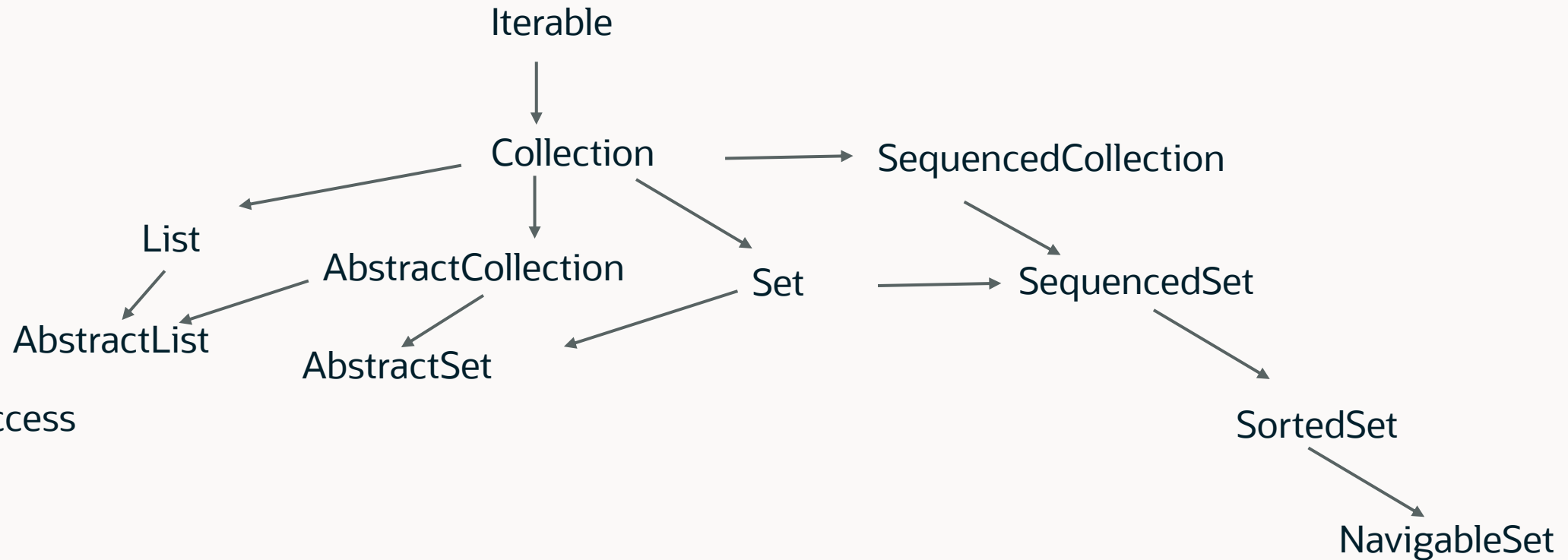
Simplified relations between Collection-related types

Additional interfaces for specialized functionality



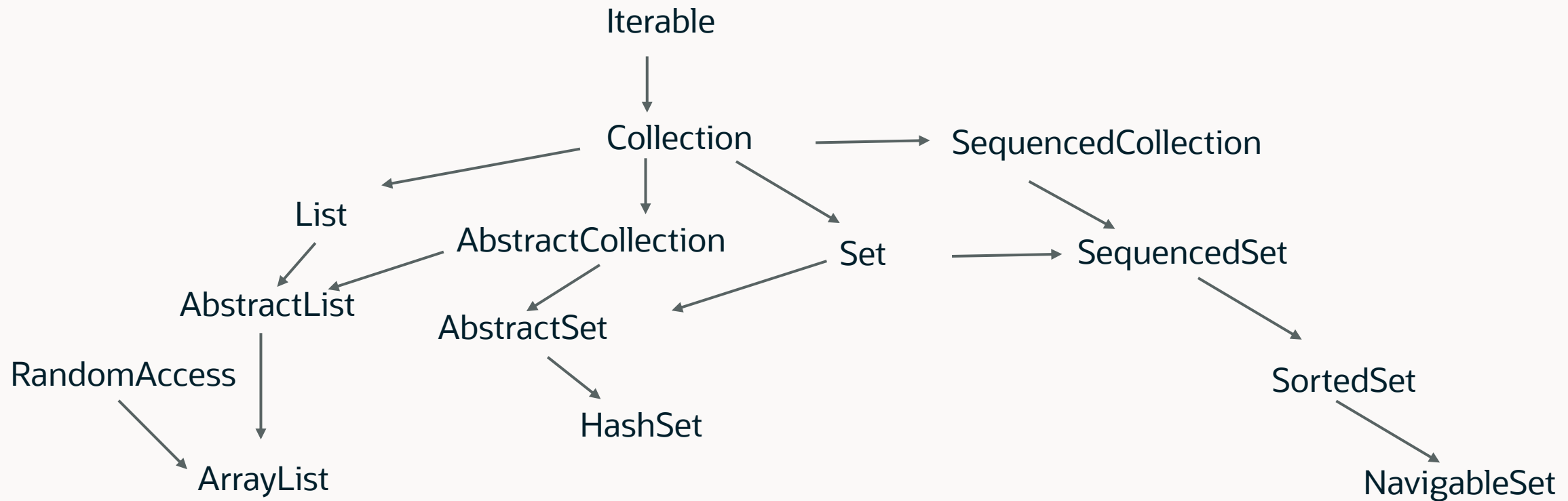
Simplified relations between Collection-related types

Marker interface to indicate implementation properties



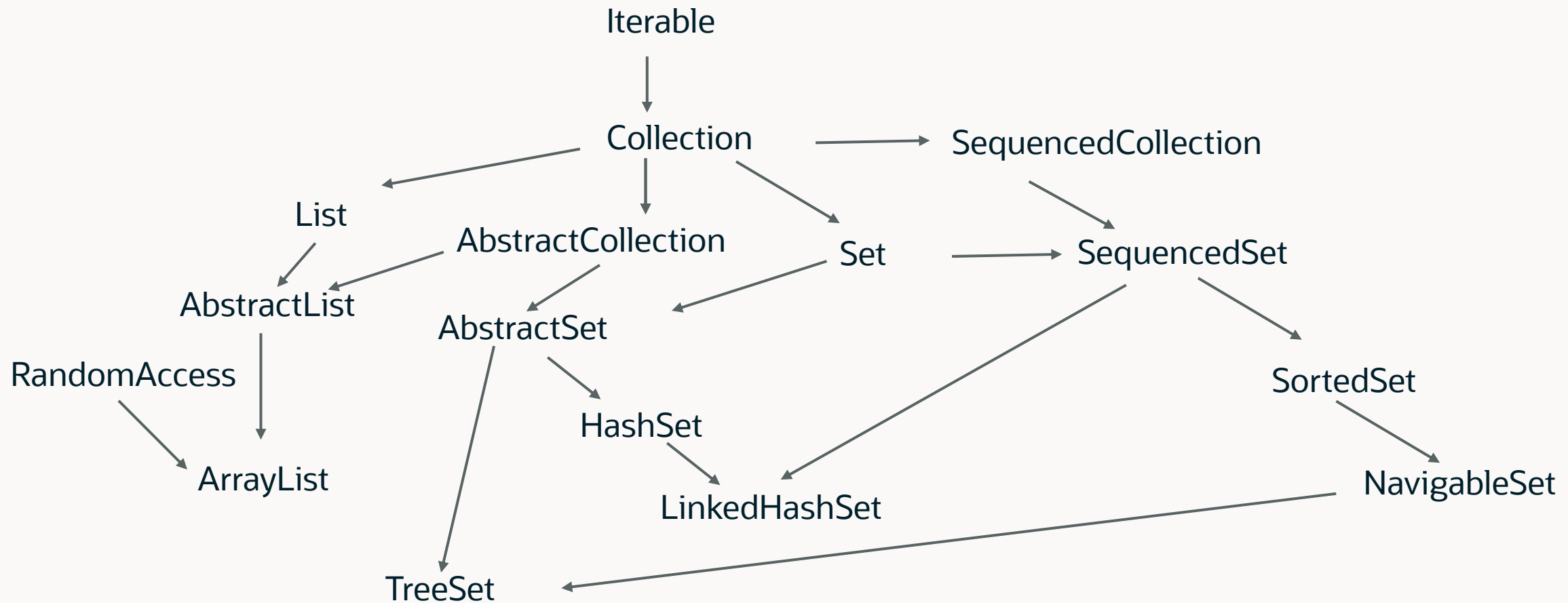
Simplified relations between Collection-related types

Commonly used concrete classes



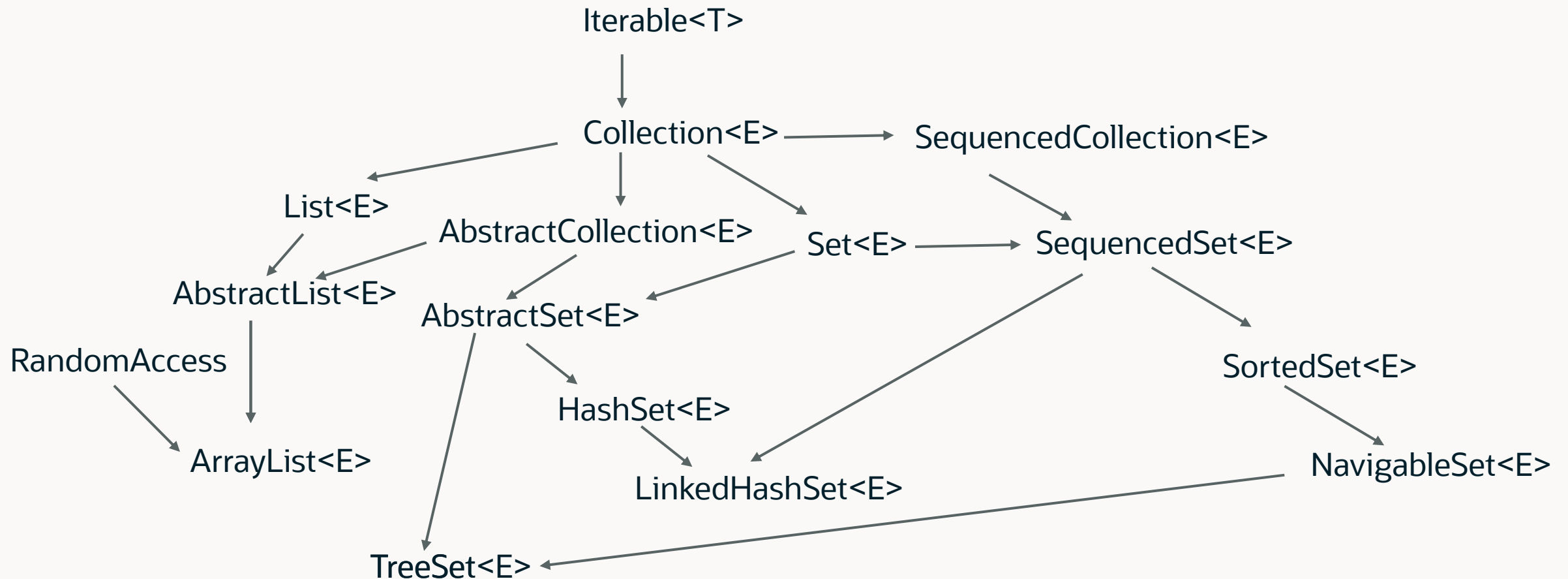
Simplified relations between Collection-related types

More specialized concrete classes



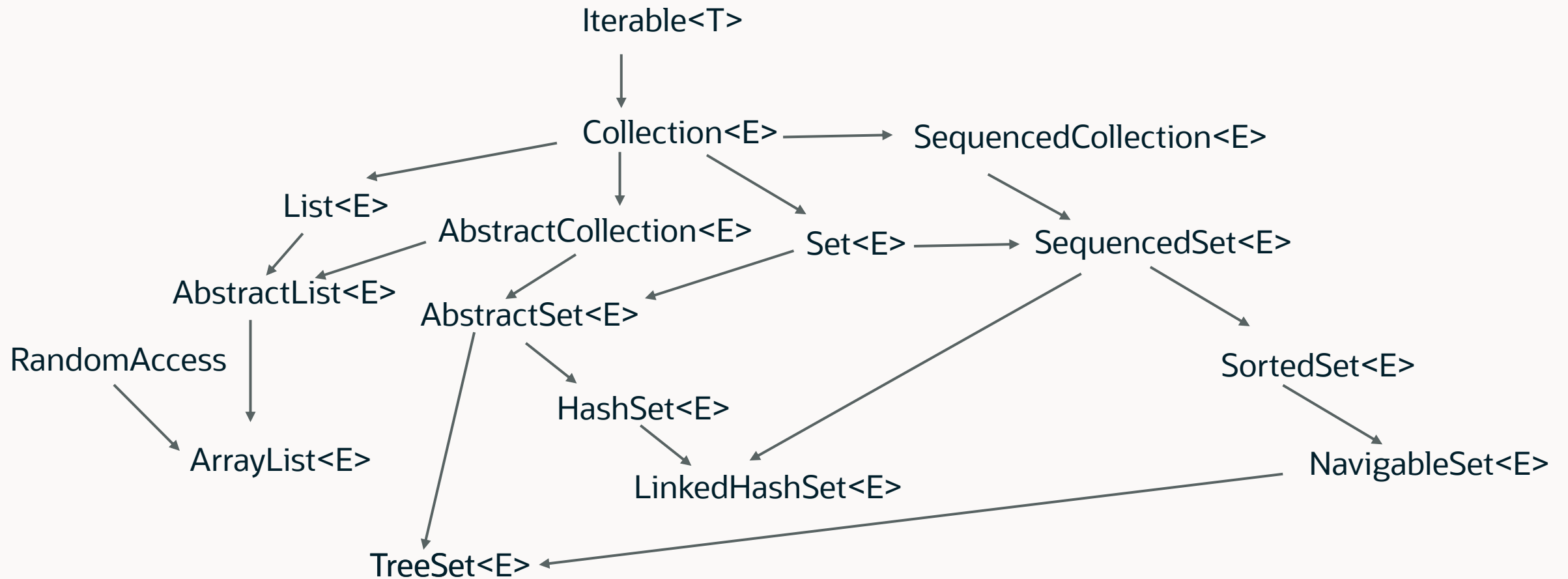
Simplified relations between Collection-related types

Generics



Simplified relations between Collection-related types

Third-party collections libraries



[Eclipse Collections](#)

Google Guava [Collections](#)

[Apache Commons Collections](#)



Any questions?

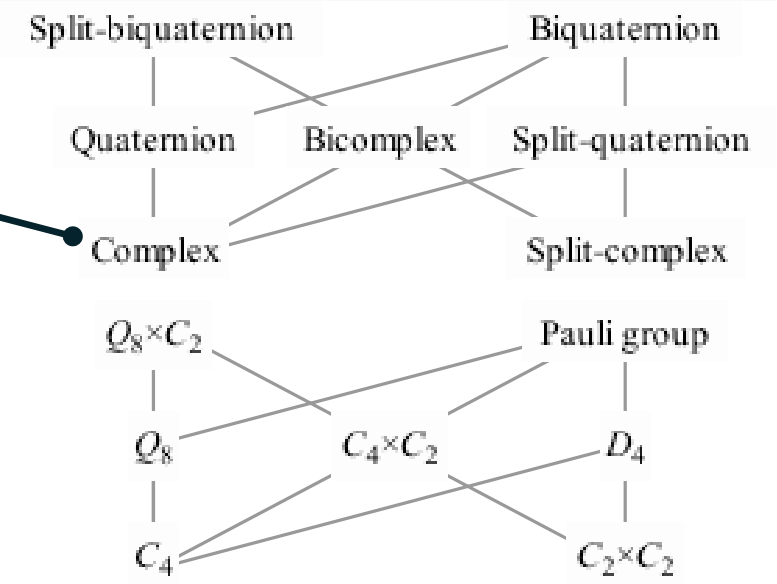
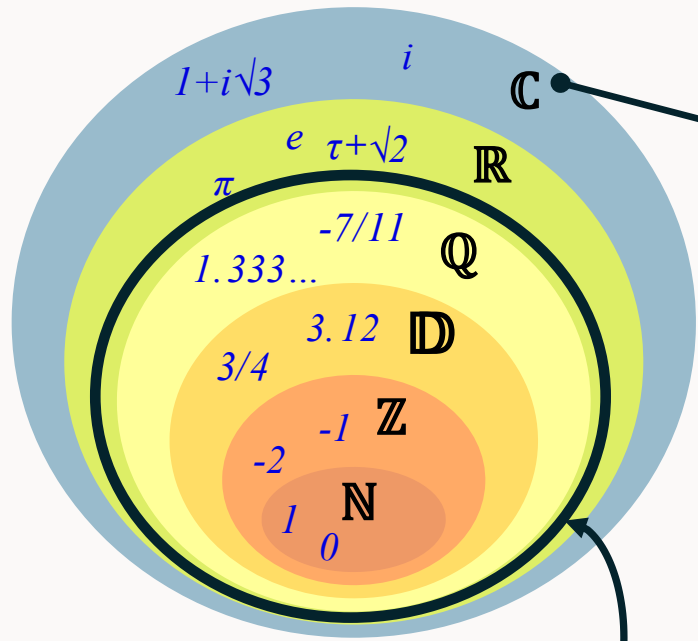
ArrayList

HashMap

Numbers

Kinds of numbers

By Cmglee - Own work, CC BY-SA 4.0, <https://commons.wikimedia.org/w/index.php?curid=166722426>

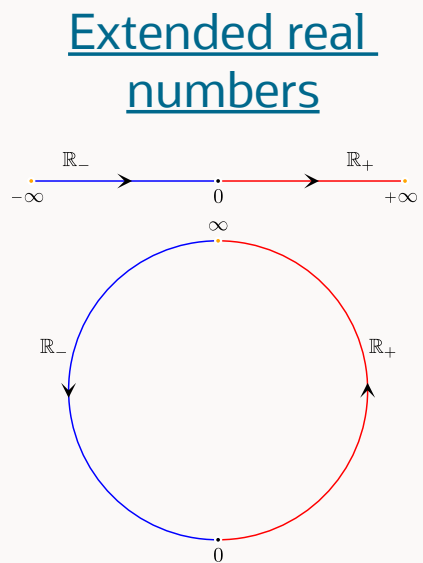


Hypercomplex numbers



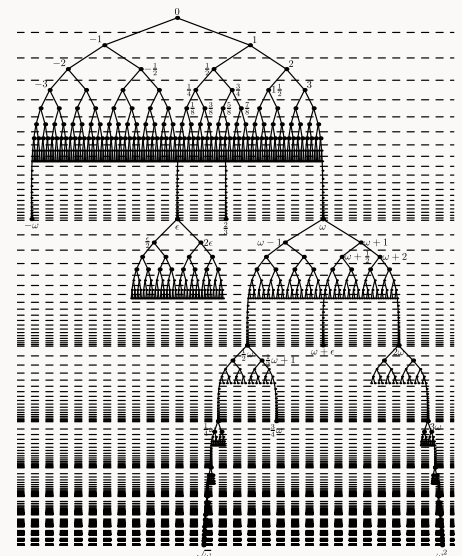
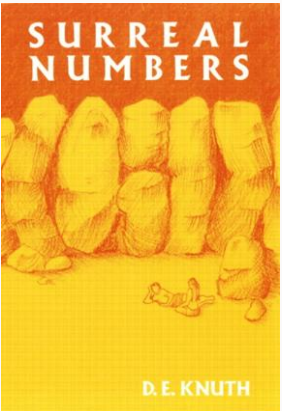
Algebraic numbers

By Stephen J. Brooks (talk) Source code in C with OpenGL. - I (Stephen J. Brooks (talk)) created this work entirely by myself., CC BY 3.0, <https://commons.wikimedia.org/w/index.php?curid=98212420>



Extended real numbers

Surreal numbers



Algebraic structures: groups, rings, and fields

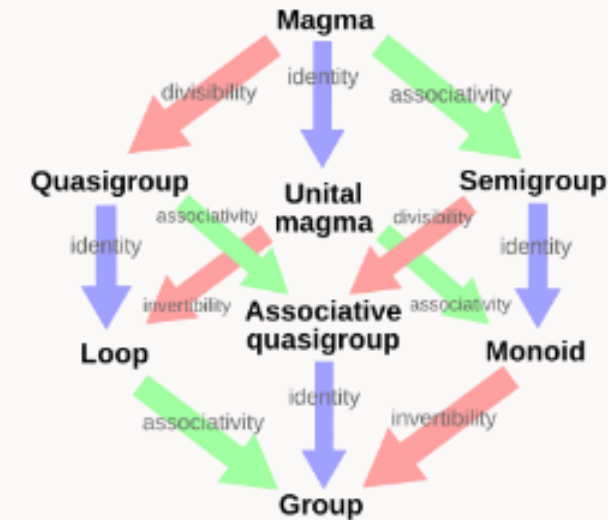
FIELD	
(A1) Closure under addition:	If a and b belong to S , then $a + b$ is also in S
(A2) Associativity of addition:	$a + (b + c) = (a + b) + c$ for all a, b, c in S
(A3) Additive identity:	There is an element 0 in R such that $a + 0 = 0 + a = a$ for all a in S
(A4) Additive inverse:	For each a in S there is an element $-a$ in S such that $a + (-a) = (-a) + a = 0$
Integral Domain	
(A5) Commutativity of addition:	$a + b = b + a$ for all a, b in S
Commutative Ring	
(M1) Closure under multiplication:	If a and b belong to S , then ab is also in S
(M2) Associativity of multiplication:	$a(bc) = (ab)c$ for all a, b, c in S
(M3) Distributive laws:	$a(b + c) = ab + ac$ for all a, b, c in S $(a + b)c = ac + bc$ for all a, b, c in S
Ring	
(M4) Commutativity of multiplication:	$ab = ba$ for all a, b in S
Abelian Group	
(M5) Multiplicative identity:	There is an element 1 in S such that $a1 = 1a = a$ for all a in S
(M6) No zero divisors:	If a, b in S and $ab = 0$, then either $a = 0$ or $b = 0$
Group	
(M7) Multiplicative inverse:	If a belongs to S and $a \neq 0$, there is an element a^{-1} in S such that $aa^{-1} = a^{-1}a = 1$

Group: $\approx$ addition and inverse elements (subtraction)

Ring: $\approx$ addition and multiplication

(Rings that aren't fields can have a division operator defined over them, but don't have multiplicative inverses the way a field does.)

Field: $\approx$ addition, subtraction, multiplication, and division



From *Cryptography and Network Security: Principles and Practice, Sixth Edition* by William Stallings

Number(-like?) entities built using numbers

Vector spaces (aka matrices, tensors, linear algebra, etc.) built using field elements

Polynomials (form a ring),

...

Integers

Real numbers

For many kinds of numbers, you can add, subtract, and multiply them, and sometimes divide them too.

Abstraction: a commonality between programming and mathematics

Many, many details and long histories in the full picture

Extensive terminology

Object oriented design and mathematical structures both involve *abstraction*

- Find commonality, even small commonality, give it a name, and see how else it can be used; example from Java, define the one-method `AutoCloseable` interface to allow use of `try-with-resources` statement
- Can make productive use of the systems *without* knowing all the terminology and implementation details. (The formal constructions of real numbers are all nontrivial.)

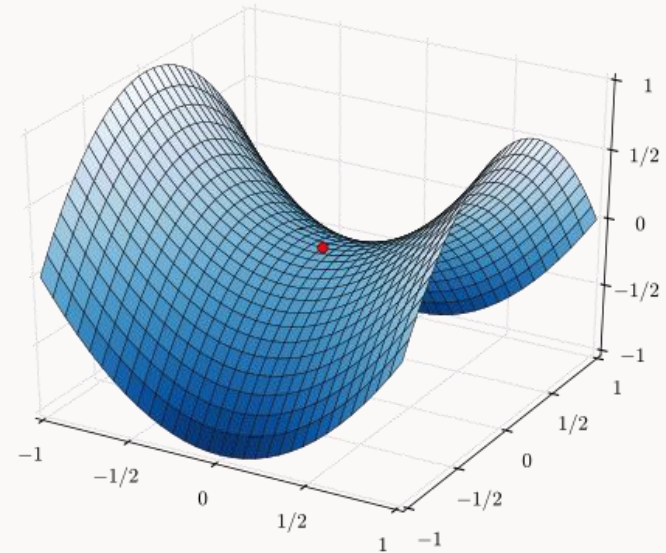
For numerical types in Java, will *not* be attempting to represent all possible mathematical structures, but will be *informed* by those structures.

Where is Java numerics situated?

Considerations include:

- Java platform, current and anticipated features.
- Mathematics
- *Computation*, separate from mathematics

Design space includes the region around the [saddle point](#) taking all these factors into account.



***Java is a blue collar language.
It's not PhD thesis material but
a language for a job.***

James Gosling
[The Feel of Java](#), Computer, June 1997.

Numeric types in the platform (java.*) as of JDK 26

Most primitive types plus `java.math.{BigInteger, BigDecimal}`

Review: what does being a primitive type get you?

- Language support
 - Operators
 - Literals
 - Compile-time constant-folding ([JLS §15.29 Constant Expressions](#))
- Works on every JVM and in all nooks and crannies of the platform
- Potential for vectorization, including auto-vectorization (see [Auto-Vectorization in HotSpot](#), JVMLS 2025)
- Dense arrays

Preferable for future numeric types to *act like* today's primitive types.

Particular numerical types of interest for different computational jobs

Inspired by, but not limited to, mathematical numbers

Complex and imaginary

Vectors/matrices

Quad (128-bit binary)

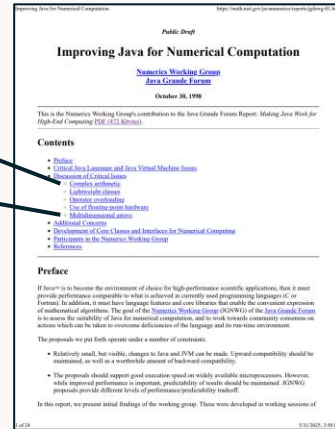
Decimal types

- IEEE 754 {32, 64, 128} bit
- SQL-style, monetary

Machine learning

- [Float16](#)
- [bfloat16](#)
- Future 8-bit floating-point types from [IEEE 3109](#) or elsewhere?

Nvidia TensorFloat-32 is a 19-bit format with the significand size of float16 and the exponent size of bfloat16. ☺



Unsigned integers (including unsigned byte)

Rational numbers/fractions

Other entries from LISP-style [numerical towers](#)

Niche applications

- [Quaternions](#) ($i^2 = j^2 = k^2 = ijk = -1$)
- [Unums/posits](#)
- [Interval arithmetics](#)
- Doubled-double
- 80-bit binary floating-point (ABI usage)
- ...



Prototypes of new numeric types

See <https://github.com/openjdk/valhalla/tree/type-classes/>

Available prototypes

- Float16 (with operator support)
- Bfloat16
- Unsigned 32-bit integers
- Exact integers (throw an exception on overflow)
- Textbook complex, Cartesian coordinates
- Textbook complex, polar coordinates
- Polynomials over double

Planned

- Decimal types
- Rational
- Unsigned integers in other sizes
- ...



How can we support all these numeric types on the Java platform?

Thought experiment of what adding a new primitive type, including JVM support, would involve.

Consider a new numeric type akin to `float` or `double`,
for concreteness keep in mind `float16` and `decimal128`.

Abstractly, how to describe numeric support for a type?

Create numeric values

- Conversion from text/strings
- Conversion from numeric values of other types
- Conversion from binary data
- ...

Operate on numeric values

- Operations appropriate for that type: e.g. $\{+, -, *, /, \dots\}$
- Library support: sqrt, sin, cos, tan, ...

Output numeric values

- Textual output, binary output
- ...

Where is that support *specified*?

Specification sections involved in float or double support.

The Java® Language Specification *Java SE 25 Edition*

James Gosling
Bill Joy
Guy Steele
Gilad Bracha
Alex Buckley
Daniel Smith
Gavin Bierman

2025-07-29

*Possible future changes:
pattern matching, etc.*

The Java® Virtual Machine Specification *Java SE 25 Edition*

Tim Lindholm
Frank Yellin
Gilad Bracha
Alex Buckley
Daniel Smith

2025-07-29

Class libraries

Module	Description
java.base	Defines the foundational APIs of the Java SE Platform.
java.compiler	Defines the Language Model, Annotation Processing, and Java Compiler APIs.
java.datatransfer	Defines the API for transferring data between and within applications.
java.desktop	Defines the AWT and Swing user interface toolkits, plus APIs for accessibility, audio, imaging, printing, and JavaBeans.
java.instrument	Defines services that allow agents to instrument programs running on the JVM.
java.logging	Defines the Java Logging API.
java.management	Defines the Java Management Extensions (JMX) API.
java.management.rmi	Defines the RMI connector for the Java Management Extensions (JMX) Remote API.
java.naming	Defines the Java Naming and Directory Interface (JNDI) API.
java.net.http	Defines the HTTP Client and WebSocket APIs.
java.prefs	Defines the Preferences API.
java.rmi	Defines the Remote Method Invocation (RMI) API.
java.scripting	Defines the Scripting API.

[JNI](#) / [FFM API](#)

[Serialization](#)/marshalling

...



JLS sections

Literals, conversions, ...

Table of Contents

1 Introduction 1

- 1.1 Organization of the Specification 2
- 1.2 Example Programs 6
- 1.3 Notation 6
- 1.4 Relationship to Predefined Classes and Interfaces 7
- 1.5 Preview Features 7
 - 1.5.1 Restrictions on the Use of Preview Features 8
 - 1.5.2 Current Preview Language Features 10
- 1.6 Feedback 10
- 1.7 References 10

2 Grammars 13

- 2.1 Context-Free Grammars 13
- 2.2 The Lexical Grammar 13
- 2.3 The Syntactic Grammar 14
- 2.4 Grammar Notation 14

3 Lexical Structure 19

- 3.1 Unicode 19
- 3.2 Lexical Translations 20
- 3.3 Unicode Escapes 21
- 3.4 Line Terminators 24
- 3.5 Input Elements and Tokens 24
- 3.6 White Space 26
- 3.7 Comments 26
- 3.8 Identifiers 28
- 3.9 Keywords 30
- 3.10 Literals 33
 - 3.10.1 Integer Literals 33
 - 3.10.2 Floating-Point Literals 40
 - 3.10.3 Boolean Literals 43
 - 3.10.4 Character Literals 44
 - 3.10.5 String Literals 45
 - 3.10.6 Text Blocks 47
 - 3.10.7 Escape Sequences 53
 - 3.10.8 The Null Literal 54
- 3.11 Separators 54
- 3.12 Operators 55

iii

The Java® Language Specification

4 Types, Values, and Variables 57

- 4.1 The Kinds of Types and Values 57
- 4.2 Primitive Types and Values 58
 - 4.2.1 Integral Types and Values 59
 - 4.2.2 Integer Operations 59
 - 4.2.3 Floating-Point Types and Values 61
 - 4.2.4 Floating-Point Operations 63
 - 4.2.5 The `boolean` Type and Boolean Values 65
- 4.3 Reference Types and Values 66
 - 4.3.1 Objects 68
 - 4.3.2 The Class `Object` 70
 - 4.3.3 The Class `String` 71
 - 4.3.4 When Reference Types Are the Same 71
- 4.4 Type Variables 72
- 4.5 Parameterized Types 74
 - 4.5.1 Type Arguments of Parameterized Types 75
 - 4.5.2 Members and Constructors of Parameterized Types 78
- 4.6 Type Erasure 79
- 4.7 Reifiable Types 79
- 4.8 Raw Types 81
- 4.9 Intersection Types 85
- 4.10 Subtyping 86
 - 4.10.1 Subtyping among Primitive Types 86
 - 4.10.2 Subtyping among Class and Interface Types 86
 - 4.10.3 Subtyping among Array Types 87
 - 4.10.4 Least Upper Bound 88
 - 4.10.5 Type Projections 91
- 4.11 Where Types Are Used 93
- 4.12 Variables 98
 - 4.12.1 Variables of Primitive Type 98
 - 4.12.2 Variables of Reference Type 99
 - 4.12.3 Kinds of Variables 101
 - 4.12.4 ~~Final~~ Variables 103
 - 4.12.5 Initial Values of Variables 105
 - 4.12.6 Types, Classes, and Interfaces 106

5 Conversions and Contexts 111

- 5.1 Kinds of Conversion 114
 - 5.1.1 Identity Conversion 114
 - 5.1.2 Widening Primitive Conversion 115
 - 5.1.3 Narrowing Primitive Conversion 116
 - 5.1.4 Widening and Narrowing Primitive Conversion 116
 - 5.1.5 Widening Reference Conversion 119
 - 5.1.6 Narrowing Reference Conversion 120
 - 5.1.6.1 Allowed Narrowing Reference Conversion 120
 - 5.1.6.2 Checked and Unchecked Narrowing Reference Conversions 122

iv

JLS sections, cont.

More conversions

The Java® Language Specification

5.1.6.3 Narrowing Reference Conversions at Run Time 123

5.1.7 Boxing Conversion 126

5.1.8 Unboxing Conversion 127

5.1.9 Unchecked Conversion 129

5.1.10 Capture Conversion 129

5.1.11 String Conversion 131

5.1.12 Forbidden Conversions 132

5.2 Assignment Contexts 132

5.3 Invocation Contexts 137

5.4 String Contexts 139

5.5 Casting Contexts 139

5.6 Numeric Contexts 145

5.7 Testing Contexts 149

6 Names 151

6.1 Declarations 152

6.2 Names and Identifiers 160

6.3 Scope of a Declaration 162

6.3.1 Scope for Pattern Variables in Expressions 166

6.3.1.1 Conditional-And Operator && 166

6.3.1.2 Conditional-Or Operator || 167

6.3.1.3 Logical Complement Operator ! 168

6.3.1.4 Conditional Operator ? : 168

6.3.1.5 Pattern Match Operator instanceof 169

6.3.1.6 switch Expressions 169

6.3.1.7 Parenthesized Expressions 169

6.3.2 Scope for Pattern Variables in Statements 169

6.3.2.1 Blocks 170

6.3.2.2 if Statements 170

6.3.2.3 while Statements 172

6.3.2.4 do Statements 172

6.3.2.5 for Statements 172

6.3.2.6 switch Statements 172

6.3.2.7 Labeled Statements 173

6.3.3 Scope for Pattern Variables in case Labels 173

6.4 Shadowing and Obscuring 173

6.4.1 Shadowing 177

6.4.2 Obscuring 180

6.5 Determining the Meaning of a Name 181

6.5.1 Syntactic Classification of a Name According to Context 182

6.5.2 Reclassification of Contextually Ambiguous Names 186

6.5.3 Meaning of Module Names and Package Names 188

6.5.3.1 Simple Package Names 188

6.5.3.2 Qualified Package Names 188

6.5.4 Meaning of PackageOrTypeNames 188

6.5.4.1 Simple PackageOrTypeNames 188

6.5.4.2 Qualified PackageOrTypeNames 189

CONVERSIONS AND CONTEXTS

Casting Contexts 5.5

Table 5.5-A. Casting to primitive types

To →	byte	short	char	int	long	float	double	boolean
From ↓								
byte	=	⊘	⊘	⊘	⊘	⊘	⊘	-
short	⊘	=	⊘	⊘	⊘	⊘	⊘	-
char	⊘	⊘	=	⊘	⊘	⊘	⊘	-
int	⊘	⊘	⊘	=	⊘	⊘	⊘	-
long	⊘	⊘	⊘	⊘	=	⊘	⊘	-
float	⊘	⊘	⊘	⊘	⊘	=	⊘	-
double	⊘	⊘	⊘	⊘	⊘	⊘	=	-
boolean	-	-	-	-	-	-	-	=
Byte	⊘	⊘	-	⊘	⊘	⊘	⊘	-
Short	-	⊘	-	⊘	⊘	⊘	⊘	-
Character	-	-	⊘	⊘	⊘	⊘	⊘	-
Integer	-	-	-	⊘	⊘	⊘	⊘	-
Long	-	-	-	-	⊘	⊘	⊘	-
Float	-	-	-	-	-	⊘	⊘	-
Double	-	-	-	-	-	-	⊘	-
Boolean	-	-	-	-	-	-	-	⊘
Object	⊘	⊘	⊘	⊘	⊘	⊘	⊘	⊘

141



JLS sections, cont.

Expressions and operators

The Java® Language Specification

- 14.18 The `throw` Statement 547
- 14.19 The `synchronized` Statement 549
- 14.20 The `try` statement 551
 - 14.20.1 Execution of `try-catch` 554
 - 14.20.2 Execution of `try-finally` and `try-catch-finally` 555
 - 14.20.3 `try-with-resources` 558
 - 14.20.3.1 Basic `try-with-resources` 560
 - 14.20.3.2 Extended `try-with-resources` 563
- 14.21 The `yield` Statement 564
- 14.22 Unreachable Statements 565
- 14.30 Patterns 572
 - 14.30.1 Kinds of Patterns 573
 - 14.30.2 Pattern Matching 576
 - 14.30.3 Properties of Patterns 577

15 Expressions 579

- 15.1 Evaluation, Denotation, and Result 579
- 15.2 Forms of Expressions 580
- 15.3 Type of an Expression 581
- 15.4 Floating-point Expressions 582
- 15.5 Expressions and Run-Time Checks 586
- 15.6 Normal and Abrupt Completion of Evaluation 587
- 15.7 Evaluation Order 589
 - 15.7.1 Evaluate Left-Hand Operand First 589
 - 15.7.2 Evaluate Operands before Operation 591
 - 15.7.3 Evaluation Respects Parentheses and Precedence 592
 - 15.7.4 Argument Lists are Evaluated Left-to-Right 593
 - 15.7.5 Evaluation Order for Other Expressions 594
- 15.8 Primary Expressions 595
 - 15.8.1 Lexical Literals 596
 - 15.8.2 Class Literals 596
 - 15.8.3 `this` 597
 - 15.8.4 Qualified `this` 598
 - 15.8.5 Parenthesized Expressions 599
- 15.9 Class Instance Creation Expressions 600
 - 15.9.1 Determining the Class being Instantiated 602
 - 15.9.2 Determining Enclosing Instances 603
 - 15.9.3 Choosing the Constructor and its Arguments 607
 - 15.9.4 Run-Time Evaluation of Class Instance Creation Expressions 611
 - 15.9.5 Anonymous Class Declarations 612
 - 15.9.5.1 Anonymous Constructors 613
- 15.10 Array Creation and Access Expressions 615
 - 15.10.1 Array Creation Expressions 615
 - 15.10.2 Run-Time Evaluation of Array Creation Expressions 616
 - 15.10.3 Array Access Expressions 619
 - 15.10.4 Run-Time Evaluation of Array Access Expressions 620

xii

The Java® Language Specification

- 15.11 Field Access Expressions 622
 - 15.11.1 Field Access Using a Primary 623
 - 15.11.2 Accessing Superclass Members using `super` 626
- 15.12 Method Invocation Expressions 627
 - 15.12.1 Compile-Time Step 1: Determine Type to Search 629
 - 15.12.2 Compile-Time Step 2: Determine Method Signature 631
 - 15.12.2.1 Identify Potentially Applicable Methods 637
 - 15.12.2.2 Phase 1: Identify Matching Arity Methods Applicable by Strict Invocation 640
 - 15.12.2.3 Phase 2: Identify Matching Arity Methods Applicable by Loose Invocation 641
 - 15.12.2.4 Phase 3: Identify Methods Applicable by Variable Arity Invocation 642
 - 15.12.2.5 Choosing the Most Specific Method 642
 - 15.12.2.6 Method Invocation Type 647
 - 15.12.3 Compile-Time Step 3: Is the Chosen Method Appropriate? 648
 - 15.12.4 Run-Time Evaluation of Method Invocation 651
 - 15.12.4.1 Compute Target Reference (If Necessary) 651
 - 15.12.4.2 Evaluate Arguments 653
 - 15.12.4.3 Check Accessibility of Type and Method 654
 - 15.12.4.4 Locate Method to Invoke 655
 - 15.12.4.5 Create Frame, Synchronize, Transfer Control 660
- 15.13 Method Reference Expressions 661
 - 15.13.1 Compile-Time Declaration of a Method Reference 664
 - 15.13.2 Type of a Method Reference 671
 - 15.13.3 Run-Time Evaluation of Method References 672
- 15.14 Postfix Expressions 676
 - 15.14.1 Expression Names 676
 - 15.14.2 Postfix Increment Operator `++` 676
 - 15.14.3 Postfix Decrement Operator `--` 677
- 15.15 Unary Operators 677
 - 15.15.1 Prefix Increment Operator `++` 679
 - 15.15.2 Prefix Decrement Operator `--` 680
 - 15.15.3 Unary Plus Operator `+` 680
 - 15.15.4 Unary Minus Operator `-` 680
 - 15.15.5 Bitwise Complement Operator `~` 681
 - 15.15.6 Logical Complement Operator `!` 681
- 15.16 Cast Expressions 682
- 15.17 Multiplicative Operators 683
 - 15.17.1 Multiplication Operator `*` 684
 - 15.17.2 Division Operator `/` 685
 - 15.17.3 Remainder Operator `%` 686
- 15.18 Additive Operators 688
 - 15.18.1 String Concatenation Operator `+` 689
 - 15.18.2 Additive Operators `+` and `-` for Numeric Types 691
- 15.19 Shift Operators 692
- 15.20 Relational Operators 694
 - 15.20.1 Numerical Comparison Operators `<`, `<=`, `>`, and `>=` 694

xiii

The Java® Language Specification

- 15.20.2 The `instanceof` Operator 695
- 15.21 Equality Operators 698
 - 15.21.1 Numerical Equality Operators `==` and `!=` 698
 - 15.21.2 Boolean Equality Operators `==` and `!=` 699
 - 15.21.3 Reference Equality Operators `==` and `!=` 700
- 15.22 Bitwise and Logical Operators 700
 - 15.22.1 Integer Bitwise Operators `&`, `^`, and `|` 701
 - 15.22.2 Boolean Logical Operators `&&`, `&`, `||`, and `||=` 702
- 15.23 Conditional-And Operator `&&` 702
- 15.24 Conditional-Or Operator `||` 703
- 15.25 Conditional Operator `?:` 704
 - 15.25.1 Boolean Conditional Expressions 711
 - 15.25.2 Numeric Conditional Expressions 711
 - 15.25.3 Reference Conditional Expressions 712
- 15.26 Assignment Operators 713
 - 15.26.1 Simple Assignment Operator `=` 714
 - 15.26.2 Compound Assignment Operators 720
- 15.27 Lambda Expressions 726
 - 15.27.1 Lambda Parameters 728
 - 15.27.2 Lambda Body 732
 - 15.27.3 Type of a Lambda Expression 735
 - 15.27.4 Run-Time Evaluation of Lambda Expressions 737
- 15.28 `switch` Expressions 738
 - 15.28.1 The Switch Block of a `switch` Expression 739
 - 15.28.2 Run-Time Evaluation of `switch` Expressions 740
- 15.29 Constant Expressions 742

16 Definite Assignment 745

- 16.1 Definite Assignment and Expressions 751
 - 16.1.1 Boolean Constant Expressions 751
 - 16.1.2 Conditional-And Operator `&&` 752
 - 16.1.3 Conditional-Or Operator `||` 752
 - 16.1.4 Logical Complement Operator `!` 752
 - 16.1.5 Conditional Operator `?:` 752
 - 16.1.6 `switch` Expressions 753
 - 16.1.7 Other Expressions of Type `boolean` 756
 - 16.1.8 Assignment Expressions 756
 - 16.1.9 Operators `++` and `--` 757
 - 16.1.10 Other Expressions 757
- 16.2 Definite Assignment and Statements 758
 - 16.2.1 Empty Statements 758
 - 16.2.2 Blocks 759
 - 16.2.3 Local Class and Interface Declarations 760
 - 16.2.4 Local Variable Declaration Statements 760
 - 16.2.5 Labeled Statements 761
 - 16.2.6 Expression Statements 761
 - 16.2.7 `if` Statements 761

xiv



One effect of conversion and promotion policies

Specification in the JLS allows a Java compiler like javac to determine how to compile the expression

`a + b;`

Based on the types of the expressions, after any necessary conversions, one of:

- Compilation error
- dadd of a and b
- iadd of a and b
- fadd of a and b
- ladd of a and b
- String concatenation sequence of a and b

Historically, could be `StringBuffer` or `StringBuilder` calls, as of JDK 9 with [JEP 280](#): “Indify String Concatenation” use `invokedynamic` via `StringConcatFactory`.

JVMS sections

Data types, class file format, constant pool structures (including annotations)

Table of Contents

1 Introduction 1

- 1.1 A Bit of History 1
- 1.2 The Java Virtual Machine 2
- 1.3 Organization of the Specification 5
- 1.4 Notation 5
- 1.5 Preview Features 6
 - 1.5.1 Restrictions on the Use of Preview Features 7
 - 1.5.2 Current Preview VM Features 8
- 1.6 Feedback 8

2 The Structure of the Java Virtual Machine 9

- 2.1 The class File Format 9
- 2.2 Data Types 10
- 2.3 Primitive Types and Values 10
 - 2.3.1 Integral Types and Values 11
 - 2.3.2 Floating-Point Types and Values 12
 - 2.3.3 The returnAddress Type and Values 13
 - 2.3.4 The boolean Type 14
- 2.4 Reference Types and Values 14
- 2.5 Run-Time Data Areas 15
 - 2.5.1 The pc Register 15
 - 2.5.2 Java Virtual Machine Stacks 15
 - 2.5.3 Heap 16
 - 2.5.4 Method Area 17
 - 2.5.5 Run-Time Constant Pool 17
 - 2.5.6 Native Method Stacks 18
- 2.6 Frames 19
 - 2.6.1 Local Variables 19
 - 2.6.2 Operand Stacks 20
 - 2.6.3 Dynamic Linking 21
 - 2.6.4 Normal Method Invocation Completion 21
 - 2.6.5 Abrupt Method Invocation Completion 22
- 2.7 Representation of Objects 22
- 2.8 Floating-Point Arithmetic 22
- 2.9 Special Methods 26
 - 2.9.1 Instance Initialization Methods 26
 - 2.9.2 Class Initialization Methods 26
 - 2.9.3 Signature Polymorphic Methods 27
- 2.10 Exceptions 28
- 2.11 Instruction Set Summary 30

iii

The Java® Virtual Machine Specification

- 2.11.1 Types and the Java Virtual Machine 31
- 2.11.2 Load and Store Instructions 33
- 2.11.3 Arithmetic Instructions 34
- 2.11.4 Type Conversion Instructions 35
- 2.11.5 Object Creation and Manipulation 37
- 2.11.6 Operand Stack Management Instructions 38
- 2.11.7 Control Transfer Instructions 38
- 2.11.8 Method Invocation and Return Instructions 38
- 2.11.9 Throwing Exceptions 39
- 2.11.10 Synchronization 39
- 2.12 Class Libraries 40
- 2.13 Public Design, Private Implementation 41
- 3 Compiling for the Java Virtual Machine 43
 - 3.1 Format Examples 43
 - 3.2 Use of Constants, Local Variables, and Control Constructs 44
 - 3.3 Arithmetic 45
 - 3.4 Accessing the Run-Time Constant Pool 46
 - 3.5 More Control Examples 51
 - 3.6 Receiving Arguments 54
 - 3.7 Invoking Methods 55
 - 3.8 Working with Class Instances 57
 - 3.9 Arrays 59
 - 3.10 Compiling Switches 61
 - 3.11 Operations on the Operand Stack 63
 - 3.12 Throwing and Handling Exceptions 63
 - 3.13 Compiling finally 67
 - 3.14 Synchronization 70
 - 3.15 Annotations 71
 - 3.16 Modules 72
- 4 The class File Format 75
 - 4.1 The ClassFile Structure 76
 - 4.2 Names 81
 - 4.2.1 Binary Class and Interface Names 81
 - 4.2.2 Unqualified Names 82
 - 4.2.3 Module and Package Names 82
 - 4.3 Descriptors 83
 - 4.3.1 Generalization 83
 - 4.3.2 Field Descriptors 83
 - 4.3.3 Method Descriptors 84
 - 4.4 The Constant Pool 86
 - 4.4.1 The CONSTANT_Class_info Structure 89
 - 4.4.2 The CONSTANT_Fieldref_info, CONSTANT_Methodref_info, and CONSTANT_InterfaceMethodref_info Structures 90
 - 4.4.3 The CONSTANT_String_info Structure 91

iv

The Java® Virtual Machine Specification

- 4.4.4 The CONSTANT_Integer_info and CONSTANT_Float_info Structures 91
- 4.4.5 The CONSTANT_Long_info and CONSTANT_Double_info Structures 92
- 4.4.6 The CONSTANT_NameAndType_info Structure 94
- 4.4.7 The CONSTANT_Utf8_info Structure 95
- 4.4.8 The CONSTANT_MethodHandle_info Structure 97
- 4.4.9 The CONSTANT_MethodType_info Structure 99
- 4.4.10 The CONSTANT_Dynamic_info and CONSTANT_InvokeDynamic_info Structures 99
- 4.4.11 The CONSTANT_Module_info Structure 100
- 4.4.12 The CONSTANT_Package_info Structure 101
- 4.5 Fields 102
- 4.6 Methods 104
- 4.7 Attributes 108
 - 4.7.1 Defining and Naming New Attributes 115
 - 4.7.2 The ConstantValue Attribute 116
 - 4.7.3 The Code Attribute 117
 - 4.7.4 The StackMapTable Attribute 120
 - 4.7.5 The Exceptions Attribute 128
 - 4.7.6 The InnerClasses Attribute 129
 - 4.7.7 The EnclosingMethod Attribute 131
 - 4.7.8 The Synthetic Attribute 133
 - 4.7.9 The Signature Attribute 134
 - 4.7.9.1 Signatures 135
 - 4.7.10 The SourceFile Attribute 139
 - 4.7.11 The SourceDebugExtension Attribute 140
 - 4.7.12 The LineNumberTable Attribute 141
 - 4.7.13 The LocalVariableTable Attribute 142
 - 4.7.14 The LocalVariableTypeTable Attribute 144
 - 4.7.15 The RuntimeVisibleAnnotations Attribute 146
 - 4.7.15.1 The annotation structure 148
 - 4.7.16 The RuntimeVisibleParameterAnnotations Attribute 152
 - 4.7.17 The RuntimeInvisibleAnnotations Attribute 151
 - 4.7.18 The RuntimeVisibleTypeAnnotations Attribute 154
 - 4.7.19 The RuntimeVisibleTypeAnnotations Attribute 155
 - 4.7.20.1 The target_info union 161
 - 4.7.20.2 The type_path structure 166
 - 4.7.21 The RuntimeInvisibleTypeAnnotations Attribute 172
 - 4.7.22 The AnnotationDefault Attribute 173
 - 4.7.23 The BootstrapMethods Attribute 174
 - 4.7.24 The MethodParameters Attribute 175
 - 4.7.25 The Module Attribute 177
 - 4.7.26 The ModulePackages Attribute 184
 - 4.7.27 The ModuleMainClass Attribute 185
 - 4.7.28 The NestHost Attribute 186
 - 4.7.29 The NestMembers Attribute 186

v



JVMS sections, cont.

Verification, runtime constant pool, etc.

The Java® Virtual Machine Specification

4.7.30	The Record Attribute	188
4.7.31	The PermittedSubclasses Attribute	189
4.8	Format Checking	191
4.9	Constraints on Java Virtual Machine Code	191
4.9.1	Static Constraints	192
4.9.2	Structural Constraints	195
4.10	Verification of class Files	199
4.10.1	Verification by Type Checking	201
4.10.1.1	Verification of Java Virtual Machine Artifacts	202
4.10.1.2	Verification Type System	206
4.10.1.3	Instruction Representation	210
4.10.1.4	Stack Map Frames and Type Transitions	212
4.10.1.5	Type Checking Methods	216
4.10.1.6	Type Checking Code Streams	220
4.10.1.7	Type Checking Load and Store Instructions	222
4.10.1.8	Type Checking for protected Members	224
4.10.1.9	Type Checking Instructions	228
4.10.2	Verification by Type Inference	348
4.10.2.1	The Process of Verification by Type Inference	348
4.10.2.2	The Bytecode Verifier	348
4.10.2.3	Values of Types long and double	351
4.10.2.4	Instance Initialization Methods and Newly Created Objects	352
4.10.2.5	Exceptions and Casting	353
4.11	Limitations of the Java Virtual Machine	355
5	Loading, Linking, and Initializing	359
5.1	The Run-Time Constant Pool	359
5.2	Java Virtual Machine Startup	362
5.3	Creation and Loading	363
5.3.1	Loading Using the Bootstrap Class Loader	365
5.3.2	Loading Using a User-defined Class Loader	366
5.3.3	Creating Array Classes	367
5.3.4	Loading Constraints	368
5.3.5	Deriving a Class from a class File Representation	369
5.3.6	Modules and Layers	373
5.4	Linking	375
5.4.1	Verification	376
5.4.2	Preparation	376
5.4.3	Resolution	377
5.4.3.1	Class and Interface Resolution	379
5.4.3.2	Field Resolution	379
5.4.3.3	Method Resolution	380
5.4.3.4	Interface Method Resolution	382
5.4.3.5	Method Type and Method Handle Resolution	384
5.4.3.6	Dynamically-Computed Constant and Call Site Resolution	388

vi

Eliding chapter 6, The Java Virtual Machine Instruction Set, sections.

- Many JVMS updates in addition to any new instructions.

JVM Class File Structures

Expand *BaseType* for Field Descriptors for new type ([JVMS §4.3.2](#))

New constant pool tags ([JVMS §4.4](#))

Constant value attribute update ([JVMS §4.7.2](#))

Stack Map updates/verifier updates ([JVMS §4.10](#))

(How many stack slots to instructions operating on the new types use?)

Annotation attribute updates, including `element_value` type tag
(assorted sections in JVMS §4.7.{[16 – 22]}.*)

...

double-related JVM instructions

- Comparisons: `dcmp<op>` (`dcmpl`, `dcmpl`)
 - Arithmetic
 - `dadd`
 - `dsub`
 - `dmul`
 - `ddiv`
 - `dneg`
 - `drem` (*not* IEEE 754 remainder)
 - Conversions:
`i2d`, `l2d`, `d2i`, `d2l`, `d2f`, `f2d`
- Loads and stores, etc.
 - `daload`, `dastore`
 - `dconst_<d>` (2 opcodes)
 - `dload`, `dload_<n>` (4 opcodes)
 - `dstore`, `dstore_<n>` (4 opcodes)
 - `dreturn`
 - `ldc2_w`

How many opcodes are used already?

3 reserved opcodes (254, 255, 202 (0xca) for breakpoints)

- About 202 allocated (0 to 201)
- Therefore, $256 - (202 + 3) = 51$ opcodes available

About 20 opcodes per type (if done like `float` or `double`) $\Rightarrow$ not many types can be added in this way.

- Could add modifiers to existing opcodes; `NARROW` to treat `float` instruction as working on `Float16`, etc.
- Various tricks could be used to reduce opcodes allocated for a type like `Float16`: after a `Float16` to `float` conversion, do the actual arithmetic using `float` instructions, and convert back, to `Float16`, etc.
- But, supporting a type like `Decimal64` or `Decimal128` directly as a primitive would require a higher opcode allocation

Limited ability to add new primitives akin to existing primitive

Another piece: class library support

“But wait, there’s more.”

Reflective support

- Core reflection: `Class.isPrimitive()`, `FOO.TYPE`, etc.
- `javax.lang.model.type.PrimitiveType`
- `java.lang.classfile` updates

Direct support: base conversion and input/output, mathematical operations, etc.

- Wrapper class like `java.lang.Double`
- Other methods in `java.lang.{Math, StrictMath}`
- `l10n`, etc.

Operations on arrays of primitives, sorting, copying, etc.

Primitive type support overview

- In the JLS and javac, literals, operators, conversions, code generation.
- In the VM, various structures to hold constants (constant pool, annotations, etc.), as well as instructions.
- In the libraries, reflective support for the type, support for mathematical operations on the type's values, and arrays of those values.
- Combination: some amount of intrinsification of library functionality

Observations on using a JVM-centric approach

Many limitations

The remaining easy bytecode space is much smaller than that needed to naturally accommodate the number of types of possible interest.

Added types in this fashion is centralized to the JDK

- Significant cross-component collaboration, javac, HotSpot, core libraries, etc.”
- Some aspects of adding new types looks to be closer than $O(n^2)$ rather than $O(n)$; e.g. JLS conversions.

Preferable to make JLS and JVMMS changes approximately once for all types rather than N times for each of N types added.

Wouldn't it be preferable if Java SE could get some new numerics types while third parties could add their own numeric types too?

**Is there another way?
(rhetorical)**

Growing a Language

Guy Steele's [keynote](#) at the 1998 ACM OOPSLA conference, [video](#)

Excerpt from *Growing a Language*

“I might say yes to [adding] each one of these [kinds of numbers to the Java programming language] , but it is clear I must say no to all of them. [...]

***To add them all to the Java programming language would be just too much.”
—Guy Steele***

Language features recommended in *Growing a Language*

To make the Java programming language more extensible, add:

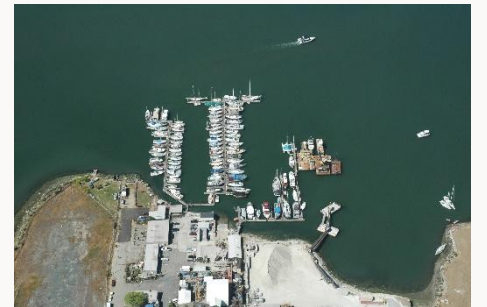
- Generics, done in Java SE 5.0 via [JSR 14](#), GA 2004
- Lightweight classes / value classes – coming with Valhalla ([JEP 401](#))
- Operator overloading

Type classes – an enabler for operator overloading...

...and conversions ... and ...

Type classes

- A language feature from our functional programming language friends; initially developed for Haskell
- Provide a way to allow opt-in mapping of an operator to a method on a class via a *witness*...
- To achieve the goal of make primitives and library-defined types inter-operate, multiple possible paths.



Numerical type support type classes

From earlier, language features of built-in numeric types include:

- Literals
- Conversions between types
- Operators
- Constant folding

Initial iterations of type classes likely to support

- Conversions between types
- Operators

Later iterations, or may not, support other features:

- Constant expressions ([JLS §15.29](#)) and constant folding at compile time
- Constants stored in the class file, including for annotation elements

How full is the glass of numeric types with type classes?

IMO, just with operators and conversions from other types, the glass would be *at least* 80% full.



Picture copyright 2025 Joseph D. Darcy All rights reserved.

What are Java's operators?

See [JLS §15 Expressions](#); group operators, in part, based on syntactic properties

Postfix increment and decrement operators {++, --}

Unary operators: “The operators +, -, ++, --, ~, !, and the cast operator ([JLS §15.16](#)) are called the *unary operators*.”

Multiplicative operators:

“The operators *, /, and % are called the *multiplicative operators*.”

Additive operators:

“The operators + and - are called the *additive operators*.”

What are Java's operators?, cont.

Relational operators:

“The numerical comparison operators <, >, <=, and >=, and the instanceof operator, are called the *relational operators*.”

Equality operators:

“The operators == (equal to) and != (not equal to) are called the *equality operators*.”

Shift operators:

“The operators << (left shift), >> (signed right shift), and >>> (unsigned right shift) are called the *shift operators*.”

Bitwise and Logical Operators:

“The bitwise operators and logical operators include the AND operator &, exclusive OR operator ^, and inclusive OR operator |.”

Other miscellaneous operators: {&&, ||, ?:, =, +=, -=, *=, /=, &=, |=, ^=, %=, <<=, >>=, >>>=}

Bundle semantically related operators together

- `java.lang.Numerical`: can use arithmetic operators
{+, -, *, /, %, - (unary), + (unary)}
- `java.lang.Orderable`: can use ordered comparison operators and min/max
{<, <=, >, >=}
- `java.lang.OrderedNumerical`: can use arithmetic AND comparison operators
(not all numbers are ordered; for example, complex numbers are *not* ordered)
- `java.lang.Integral`, extends `OrderedNumerical`: adds integer-specific operators, shifts, etc.
Adds {&, |, ^, ~, <, >>, >>>}
- `java.lang.StandardFloatingPoint`, extends `OrderedNumerical`, includes various additional methods

Excerpt from Numerical

```
public interface Numerical<NT> {  
    /**  
     * Addition operation, binary operator "{@code +}".  
     * ...  
     */  
    NT add(NT addend, NT augend);  
  
    /**  
     * Subtraction operation, binary operator "{@code -}".  
     *  
     * @implSpec  
     * The default implementation returns the sum of the first  
     * operand with the negation of the second operand.  
     * ...  
     */  
    default NT subtract(NT minuend, NT subtrahend) {  
        return this.add(minuend, this.negate(subtrahend));  
    }  
}
```

Use of a witness in UnsignedInteger



```
private static final Integral<UnsignedInt> INT = new Integral<UnsignedInt>() {  
    public UnsignedInt add(UnsignedInt addend, UnsignedInt augend) {  
        return UnsignedInt.add(addend, augend);  
    }  
  
    @Override  
    public UnsignedInt subtract(UnsignedInt minuend, UnsignedInt subtrahend) {  
        return UnsignedInt.subtract(minuend, subtrahend);  
    }  
  
    public UnsignedInt multiply(UnsignedInt multiplier, UnsignedInt multiplicand) {  
        return UnsignedInt.multiply(multiplier, multiplicand);  
    }  
}
```

Expected pattern: *Instance* method of the interface is implemented by calling *static* method on the numerical type.

```
...  
/**  
 * Witness for the {@code Numerical} interface.  
 */  
public __witness Numerical<UnsignedInt> NUM = INT;  
...  
}
```

Some syntax to declare the witness

Tells javac what it needs to do to compile an operator

Using subject-to-change set of language specification rules, javac knows:

- Where to look for witnesses
- Once found, which method on the witness to call
- Uses code generation idioms that are easy to digest for HotSpot, inlining and intrinsification friendly (analogous to newer code generation idiom for string concatenation)

Envisioned update allowing Float16 operators

Upgrading some numeric types to use operators may be simple

Change the type declaration to a value class

Add a witness declaration to the class:

```
-public final class Float16
+public final value class Float16
    extends Number
    implements Comparable<Float16> {

+    private static final StandardFloatingPoint<Float16> SFP = new
StandardFloatingPoint<Float16>() {
+        public Float16 add(Float16 addend, Float16 augend) {
+            return Float16.add(addend, augend);
+        }
...
+    /**
+     * Witness for the {@code Numerical} interface.
+     */
+    public __witness Numerical<Float16> NUM = SFP;
```

A witness for an interface with more methods would be longer, etc.

Expect *instance* methods on the witness implementation to call *static* methods of the numeric type.



Source code to class file (prototype)



For source code like:

```
Float16 a = Float16.valueOf("1.0");  
Float16 b = 2.0f // future type class conversion  
Float16 sum = a + b;
```

generated class file instructions for the + operation:

```
11: ldc          #18          // Dynamic #0: __witness:Ljava/lang/Numerical;  
13: aload_1  
14: aload 2  
15: invokeinterface #22, 3      // InterfaceMethod java/lang/Numerical.add:(Ljava/lang/Object;Ljava/lang/Object;)Ljava/lang/Object;  
(If the feature was being done pre-indy/condy, would use a different translation.)
```

Get the witness object.

Call the “add” method on the witness object.

Kind of witness used for prototypes of new numeric types

Available prototypes

- Float16: StandardFloatingPoint witness
- Bfloat16: StandardFloatingPoint witness
- UnsignedInt: Integral witness
- Exact integers: Integral witness
- Textbook complex, Cartesian coordinates: Numerical witness (complex numbers are *not* ordered)
- Textbook complex, polar coordinates: Numerical witness (complex numbers are *not* ordered)
- Polynomials over double: Numerical witness

Planned

- Decimal types: Numerical or StandardFloatingPoint witness
- Rational: Numerical witness
- Unsigned integers in other sizes: Integral witness
- ...

Type classes and operator overloading

Working design assumptions of the language feature

Assume homogenous typing

$$\alpha \text{ op } \alpha \rightarrow \alpha$$

is sufficient for operator overloading

- In other words, *do* not attempt (initially) to support mixed-type operators
- Some of the effect of mixed-type operators can be implemented via conversions

Assume only identity-less classes, that is value classes, will be eligible for operator overloading

Assume not all operators will be eligible for overloading. Some compound operators *may* at most be overloaded indirectly via a compiler-generated composite of a non-compound overload; e.g. += built out of +.



Implications of assumptions

Homogeneity does not necessarily provide ideal support for

- Complex arithmetic
(would benefit from direct `double op complex`, `imaginary op complex` and `imaginary * imaginary → double`, `imaginary + double → complex`, etc.
Possible, if costly, to implement these more precise checks inside a complex type.)
- Matrices
Matrices can be multiplied together *and* multiplied by a scalar of the type of the entities of the matrix. If only `Matrix * Matrix` multiply is supported, `1×1` matrix and a scalar are distinct concepts, but systems can work out conversions/coercions between the two.

Float16 Retrospective and Prospective

Float16 Functionality in the JDK in JDK 26

`jdk.incubator.vector.Float16` as of JDK 24

`float16` is a 16-bit IEEE 754 floating-point format (called “binary16” in the standard); cf. `bfloat16` from Google which is not covered by a standards body).

`Float16` in the incubator provides a fully-functional software implementation

- Favored simple, easy-to-understand code that doesn’t perform excessively poorly

HotSpot intrinsifications in mainline JDK 25 for x64, aarch64, and RISC-V;

thanks to OpenJDK collaborators, especially Intel being on the vanguard of adding this support!

Demo of Float16 with operators on type-classes branch



Float16 by the numbers

Work originally done in the base module on a Valhalla branch; ported to `jdk.incubator.vector` in JDK 24.

Initial push:

- $\approx$ 2,500 lines of code for Float16 (including comments and specification)
- $\approx$ 100 LOC of supporting Float16Consts
- $\approx$ 1,000 LOC for regression tests

Did *not* add...

Format-specific sin, cos, tan, ...

- For now, could use double version of those methods, e.g.
`Float16 sinOf_f16 = Float16.valueOf(Math.sin(f16.doubleValue()))`
- Do not have these methods specific for float either (unlike the C math library)

Easy things that went as expected

Can a wider floating-point format easily emulate the results of a narrower floating-point format. Sometimes yes!

Numerical analysis: potential hazards of *double-rounding* avoided when the **$2p + 2$** property holds for two formats and the $\{+, -, *, /\}$ and sqrt operations

- Can implement operations in the narrower format by converting to the wider format, doing the arithmetic in the wider format, and converting back to the narrow format
- For IEEE 754-style types, holds for both binary and decimal bases.
- Citations of this property included in comments of `Float16.java`.

Simple implementation of an arithmetic operation:

```
public static Float16 add(Float16 addend, Float16 augend) {  
    return valueOf(addend.floatValue() + augend.floatValue());  
}
```

Small things that took longer than expected: fma saga

A richness of numerical analysis

Explanation of implementation

Method specification

```
/**
 * Returns the fused multiply add of the three arguments; that is,
 * returns the exact product of the first two arguments summed
 * with the third argument and then rounded once to the nearest
 * {@code Float16}.
 *
 * The handling of zeros, NaN, infinities, and other special cases
 * by this method is analogous to the handling of those cases by
 * {@link Math#fma(float, float, float)}.
 *
 * @apiNote This method corresponds to the fusedMultiplyAdd
 * operation defined in IEEE 754.
 *
 * @param a a value
 * @param b a value
 * @param c a value
 *
 * @return (<i>a</i> * <i>b</i>) + <i>c</i> computed, as if with unlimited range and precision, and rounded
 * once to the nearest {@code Float16} value
 *
 * @see Math#fma(float, float, float)
 * @see Math#fma(double, double, double)
 */
```

```
public static Float16 fma(Float16 a, Float16 b, Float16 c) {
    * The double format has sufficient precision that a Float16
    * fma can be computed by doing the arithmetic in double, with
    * one rounding error for the sum, and then a second rounding
    * error to round the product-sum to Float16. In pseudocode,
    * this method is equivalent to the following code, assuming
    * casting was defined between Float16 and double:
    *
    * double product = (double)a * (double)b; // Always exact
    * double productSum = product + (double)c;
    * return (Float16)productSum;
    *
    * (Note that a similar relationship does not hold between
    * the double format and computing a float fma.)
    *
    * Below is a sketch of the proof that simple double
    * arithmetic can be used to implement a correctly rounded
    * Float16 fma.
    *
    * -----
    *
    * As preliminaries, the handling of NaN and Infinity
    * arguments falls out as a consequence of general operation
    * of non-finite values by double * and +. Any NaN argument to
    * fma will lead to a NaN result, infinities will propagate or
    * get turned into NaN as appropriate, etc.
    *
    * One or more zero arguments are also handled correctly,
    * including the propagation of the sign of zero if all three
    * arguments are zero.
    *
    * The double format has 53 logical bits of precision and its
    * exponent range goes from -1022 to 1023. The Float16 format
    * has 11 bits of logical precision and its exponent range
    * goes from -14 to 15. Therefore, the individual powers of 2
    * representable in the Float16 format range from the
    * subnormal 2-24, MIN_VALUE, to 215, the leading bit
    * position of MAX_VALUE.
    *
    * In cases where the numerical value of (a * b) + c is
    * computed exactly in a double, after a single rounding to
    * Float16, the result is necessarily correct since the one
    * double -> Float16 conversion is the only source of
    * numerical error. The operation as implemented in those
    * cases would be equivalent to rounding the infinitely precise
    * value to the result format, etc.
```

```

    * However, for some inputs, the intermediate product-sum will
    * not be exact and additional analysis is needed to justify
    * not having any corrective computation to compensate for
    * intermediate rounding errors.
    *
    * The following analysis will rely on the range of bit
    * positions representable in the intermediate
    * product-sum.
    *
    * For the product a*b of Float16 inputs, the range of
    * exponents for nonzero finite results goes from 2-(48)
    * (from MIN_VALUE squared) to 231 (from the exact value of
    * MAX_VALUE squared). This full range of exponent positions,
    * (31 - (-48) + 1) = 80 exceeds the precision of
    * double. However, only the product a*b can exceed the
    * exponent range of Float16. Therefore, there are three main
    * cases to consider:
    *
    * 1) Large exponent product, exponent > Float16.MAX_EXPONENT
    *
    * The magnitude of the overflow threshold for Float16 is:
    * MAX_VALUE + 1/2 *ulp(MAX_VALUE) = 0x1.fcfp15 + 0x0.002p15 = 0x1.fcfp15
    *
    * Therefore, for any product greater than or equal in
    * magnitude to 0x1.fcfp15 + MAX_VALUE = 0x1.fcfp16, the
    * final fma result will certainly overflow to infinity (under
    * round-to-nearest) since adding in c = -MAX_VALUE will still
    * be at or above the overflow threshold.
    *
    * If the exponent of the product is 15 or 16, the smallest
    * subnormal Float16 is 2-24 and the ~40 bit wide range bit
    * positions would fit in a single double exactly.
    *
    * 2) Exponent of product is within the range of normal
    * Float16 values; Float16.MIN_EXPONENT <= exponent <= Float16.MAX_EXPONENT
    *
    * The exact product has at most 22 contiguous bits in its
    * logical significand. The third number being added in has at
    * most 11 contiguous bits in its significand and the lowest
    * bit position that could be set is 2-(24). Therefore, when
    * the product has the maximum in-range exponent, 215, a
    * single double has enough precision to hold down to the
    * smallest subnormal bit position, 15 - (-24) + 1 = 40 <
    * 53. If the product was large and rounded up, increasing the
    * exponent, when the third operand was added, this would
    * cause the exponent to go up to 16, which is within the
    * range of double, so the product-sum is exact and will be
    * correct when rounded to Float16.
    *
    * 3) Exponent of product is in the range of subnormal values or smaller,
    * exponent < Float16.MIN_EXPONENT
    *
    * The smallest exponent possible in a product is 2-(48).
    * For moderately sized Float16 values added to the product,
    * with an exponent of about 4, the sum will not be
    * exact. Therefore, an analysis is needed to determine if the
    * double-rounding is benign or would lead to a different
    * final Float16 result. Double rounding can lead to a
    * different result in two cases:
    *
    * 1) The first rounding from the exact value to the extended
    * precision (here 'double') happens to be directed toward_0
    * to a value exactly midway between two adjacent working
    * precision (here 'Float16') values, followed by a second
    * rounding from there which again happens to be directed
    * toward_0 to one of these values (the one with lesser
    * magnitude). A single rounding from the exact value to the
    * working precision, in contrast, rounds to the value with
    * larger magnitude.
    *
    * 2) Symmetrically, the first rounding to the extended
    * precision happens to be directed away from 0 to a value
    * exactly midway between two adjacent working precision
    * values, followed by a second rounding from there which
    * again happens to be directed away from 0 to one of these
    * values (the one with larger magnitude). However, a single
    * rounding from the exact value to the working precision
    * rounds to the value with lesser magnitude.
    *
    * If the double rounding occurs in other cases, it is
    * innocuous, returning the same value as a single rounding to
    * the final format. Therefore, it is sufficient to show that
    * the first rounding to double does not occur at the midpoint
    * of two adjacent Float16 values:
    *
    * 1) If a, b and c have the same sign, the sum a*b + c has a
    * significant with a large gap of 20 or more 0s between the
    * bits of the significand of c to the left (at most 11 bits)
    * and those of the product a*b to the right (at most 22
    * bits). The rounding bit for the final working precision of
    * 'float16' is the leftmost 0 in the gap.
    *
    * a) If rounding to 'double' is directed toward 0, all the
    * 0s in the gap are preserved, thus the 'float16' rounding
    * bit is unaffected and remains 0. This means that the
    * 'double' value is not the midpoint of two adjacent
    * 'float16' values, so double rounding is harmless.
    *
    * b) If rounding to 'double' is directed away from 0, the
    * rightmost 0 in the gap might be replaced by a 1, but the
    * others are unaffected, including the 'float16' rounding
    * bit. Again, this shows that the 'double' value is not
    * the midpoint of two adjacent 'float16' values, and double
    * rounding is innocuous.
    *
    * 2) If a, b and c have opposite signs, in the sum a*b + c
    * the long gap of 0s above is replaced by a long gap of
    * 1s. The 'float16' rounding bit is the leftmost 1 in the
    * gap, or the second leftmost 1 if c is a power of 2. In
    * both cases, the rounding bit is followed by at least
    * another 1.
    *
    * a) If rounding to 'double' is directed toward 0, the
    * 'float16' rounding bit and its follower are preserved and
    * both 1, so the 'double' value is not the midpoint of
    * two adjacent 'float16' values, and double rounding is
    * harmless.
    *
    * b) If rounding to 'double' is directed away from 0, the
    * 'float16' rounding bit and its follower are either
    * preserved (both 1), or both switch to 0. Either way, the
    * 'double' value is again not the midpoint of two
    * adjacent 'float16' values, and double rounding is
    * harmless.
    */
```

```

    * product is numerically exact in float before the cast to
    * double; not necessary to widen to double before the
    * multiply
    double product = (double)(a.floatValue()) * b.floatValue();
    return valueOf(product + c.doubleValue());
}
```

Implementation

HT Raffaello Giuliatti for helping to work out the explanation details.



A way to implement Float16.fma with less thinking

Screen for NaN, infinity, and signed zero arguments, then for most Float16 values a, b, and c use something like

```
Float16.valueOf(new BigDecimal(a.floatValue() * b.floatValue())).add(c.floatValue())
```

The BigDecimal to Float16 conversion is nontrivial since the $2p + 2$ property does not hold for that operation, meaning that a conversion chain

BigDecimal $\rightarrow$ float $\rightarrow$ Float16

is *not* equivalent to

BigDecimal $\rightarrow$ Float16.

(The above was not used as a transitory implementation because we didn't implement the BigDecimal $\rightarrow$ Float16 conversion before implementing fma.)

Bigger-than-small items that took effort

BigDecimal ↔ Float16 conversions; many operations can have a correct-if-slow implementation operating in an “exact” type and then rounding once to the final precision

Properly rounding base conversion

- Shortest-string binary → decimal conversion is dependent not only on the numerical value but also on the precision of the *format*
- Decimal → binary conversion dependent on precision too ($2p + 2$ doesn't work in all cases for base conversion)

Consider implementing a narrow binary floating-point type

Such as bfloat or the logically 19-bit TensorFloat-32

Basic arithmetic operators should be straightforward.

Some operators like fma may need more effort.

Correctly-rounded binary $\leftrightarrow$ decimal base conversion can be tricky.

Possible platform infrastructure for adding numerical types

Including outside of the platform

To ease such writing additional binary floating-point types, candidate platform infrastructure :

- A `BigBinary` class to host higher-precision values or ease analysis.
Could be just even if only to support initial implementations/bootstrapping, it doesn't necessarily have to be super fast.
- `BigDecimal` and `BigBinary` operations that round at *exponent* boundaries too.
- Include `java.lang.runtime`-esque support for decimal ↔ binary conversion at different precisions
i.e. don't expect users to have implement “shortest string” semantics themselves.

Suggest authors start with conversion to/from `Big{$BASE}` as an early or bootstrapping step, gives a lot of functionality, at possibly slow performance, akin to the Zero interpreter in HotSpot.



Current thoughts on modeling numerics: additional design considerations

What about `java.lang.Number`?

Very little “numeric” about `java.lang.Number`

Operationally just means “can be converted to a primitive type”; doesn’t define any operations on the “number” type per se.

Number specification doesn’t strictly require results of the different *primitiveValue()* conversion methods to be consistent; e.g. `myNumber.intValue()` is *not* required to be equal to `(int)(myNumber.doubleValue())`.

Would be better as an interface rather than an abstract class (slow interface dispatch on early JVM implementations?)

Not a solid foundation for future work

API design point: unsigned integers and integers

What about operations like unary negation?

One approach for unary negation, from [JLS §15.15.4](#):

“For integer values, negation is the same as subtraction from zero.”

If negation is defined for unsigned integers, should it be defined this way? How about unconditionally throwing an exception?

If negation is *not* defined for unsigned integers, don't have to worry about this detail.

Design philosophies: Lumpers vs Splitters (a tale [as old as taxonomies](#))

- Splitter solution: `UnsignedIntegral` a separate superinterface of `Integral`
- Lumper solution: single `Integral` used by both signed and unsigned integral classes

Design considerations in other potential new numeric types: complex and imaginary numbers

Numeric considerations in addition to language usage ones.

Why look at complex numbers?

[JDK-8328365](#): Investigate adding complex numbers support to the platform

Useful for engineering calculations (electricity, etc.)

Useful mathematically, complex numbers are needed to represent all the solutions to polynomials with rational coefficients

(i.e. fundamental theorem of algebra, a degree n polynomial will have n roots, counted with multiplicity, etc.)

Commonly used an introductory example for abstract data types, *but* more advanced usage pushes on language and library support.

CS 101 design considerations for a complex number type

As an abstract data type

Assume the textbook arithmetic formulas are in use

How to model a complex number: `class`, `record`, `value class`?

- `final` vs non-`final` on the type-level
- Access level of methods, `static` vs instance declaration of particular methods
- Constructors vs. factories, etc.

Should the model be real and imaginary components or magnitude and phase/angle components?

...

Different categories of design issues

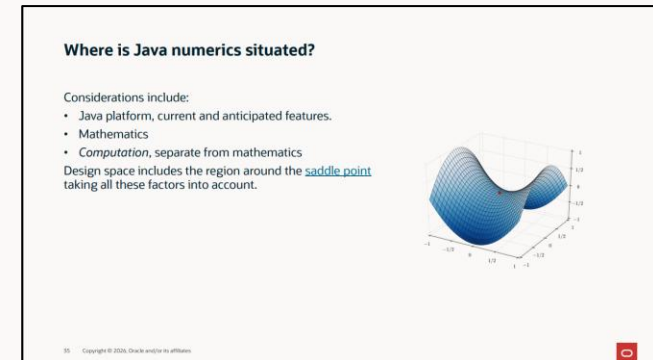
Not quite independent categories of concern

Language and platform support issues and touch points

- Operator overloading:
 - Is an ordered impose to allow comparison operators?
 - Is an operator defined for complex conjugation?

Weighing considerations: ease of use / utility of complex computation vs platform complexity

- Conversions: is there a distinct imaginary type?
- Literals
- Constant folding
- Pattern matching
- Relation to complex support on other platforms, including C, C#, Swift, etc.



Different categories of design issues, cont.

Not quite independent categories of concern

Just as there is real analysis for real numbers, there is complex analysis for complex numbers.

Modeling issues

- How are infinities added to complex numbers?
- How is NaN represented?
- Is the end goal `Complex`, or `Complex<double>` with separate `Complex<float>` and perhaps `Complex<StandardFloatingPoint>`?

Numerical Issues

- Extent of math library support (`sin`, `cos`, `tan`, ...) and [branch cuts](#)
- Defining error bounds and semantics for arithmetic operations

Example: complex multiply

Minimal javadoc spec: “Returns the product of the complex arguments.”

Textbook formula: given $(a + b \cdot i)$ and $(c + d \cdot i)$, their product is:

$$(ac - bd) + (ad + bc) \cdot i$$

Three floating-point operations to compute each component, *but*

- “*There be (numerical) dragons.*” Cancellation, overflow, etc., even without considering special value handling.
- How accurate is the computed product in a norm-wise sense?
- How accurate are the computed real and imaginary components of the product compared to the exact real and imaginary components?
- Can users expect $(c1 * c2)$ and $(c2 * c1)$ to be equivalent?

There are formulas with high per-component accuracy and commutative multiply, at the cost of additional floating-point ops, including fma.

Accurate, commutative complex multiply

See *Handbook of Floating-Point Arithmetic*, 2nd edition, Muller et al., section 11.3

```
// product's real      component is accurate_ad_bc(a, b, c, d);
// product's imaginary component is accurate_ad_bc(a, d, b, -c);

private static double accurate_ad_bc(double a, double b, double c, double d) {
    p1 = a * d;
    e1 = fma(a, d, -p1); // a*d - p1

    p2 = b * c;
    e2 = fma(b, c, -p2); // b*c - p2

    p = p1 - p2;
    e = e1 - e2;
    return p + e;
}
```

Find the round-off error in computing each product and add that back into the final result to reduce overall error.

Depending on how you count, 7 to 9 floating-point operations to compute each component (fma is often counted as two floating-point operations).

Goal: users of the Java platform should have few occasions to need to understand the error behavior of complex arithmetic operations

Have the operations easy to use and accurate enough and fast enough by default.

Summary

Many numeric types of interest for practical programming tasks.

Platform support needed for good integration

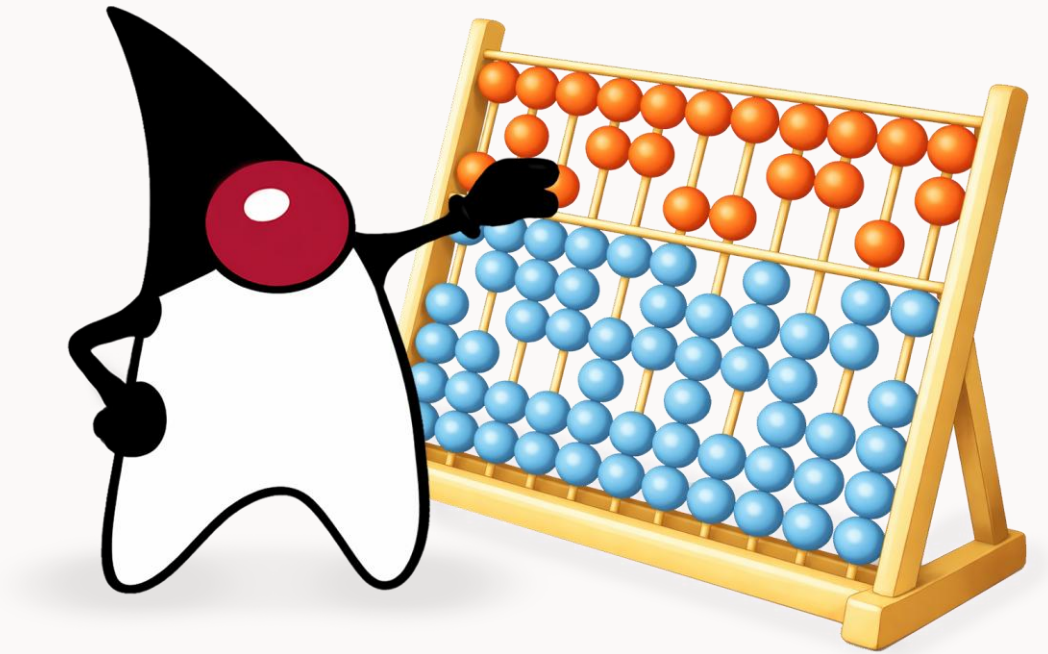
Project Valhalla features in use in numerical prototypes

Much more to do!

- Interesting conversions to explore among existing prototypes
- More prototypes expected
- Refinements and evolution to the API likely

Fun times ahead!





Q &A

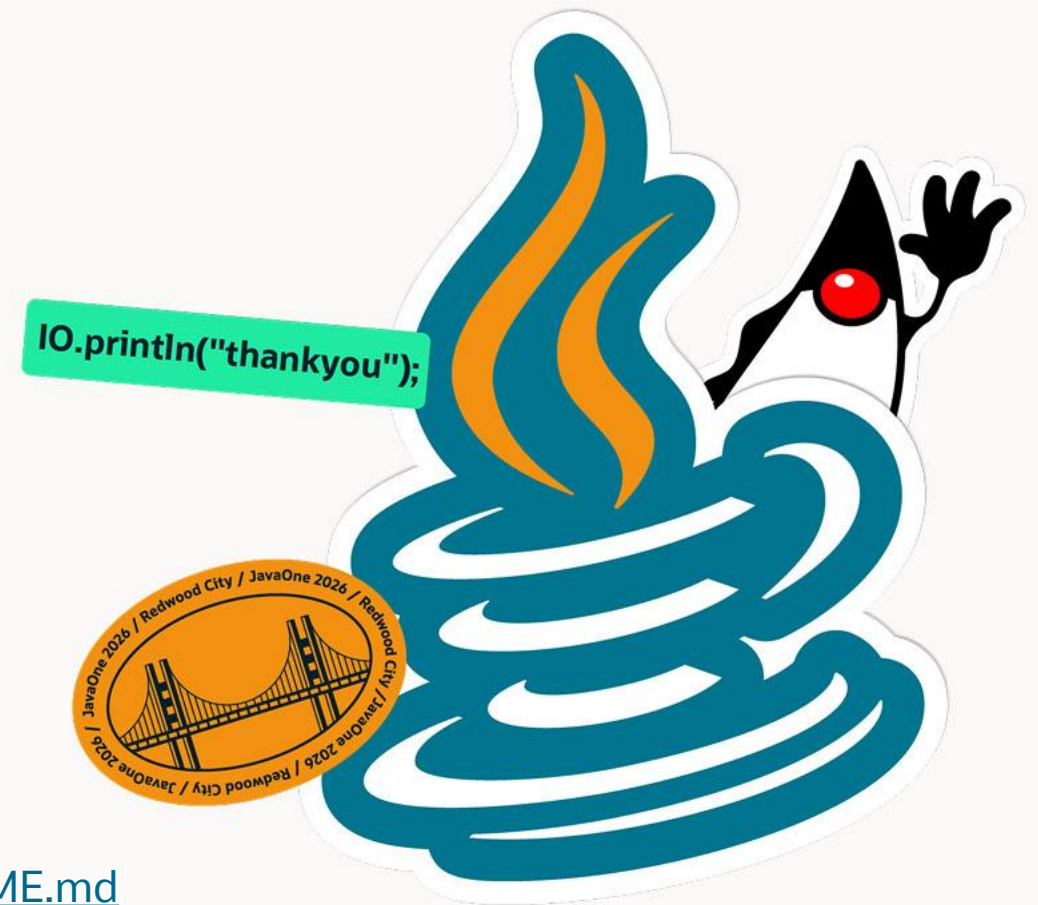
Slides will be posted to:

<https://github.com/jddarcy/SpeakingArchive/blob/master/README.md>

Thank you

Slides will be posted to:

<https://github.com/jddarcy/SpeakingArchive/blob/master/README.md>



Appendix

Implications for BigInteger and BigDecimal

The BigInteger and BigDecimal classes are non-final and have public constructors and there are known to be subclasses in use for understandable and valid reasons.

“It was not widely understood that immutable classes had to be effectively final when BigInteger and BigDecimal were written...”

Effective Java, 1st edition, Item 13: Favor Immutability, 2001

Challenging to migrate these classes to *not* have identity within the JDK’s existing compatibility policies and practices.

Acknowledged to be very desirable for the platform to have arbitrary precision integer and decimal arithmetic usable with operators.

Speculative indicators for algebraic properties

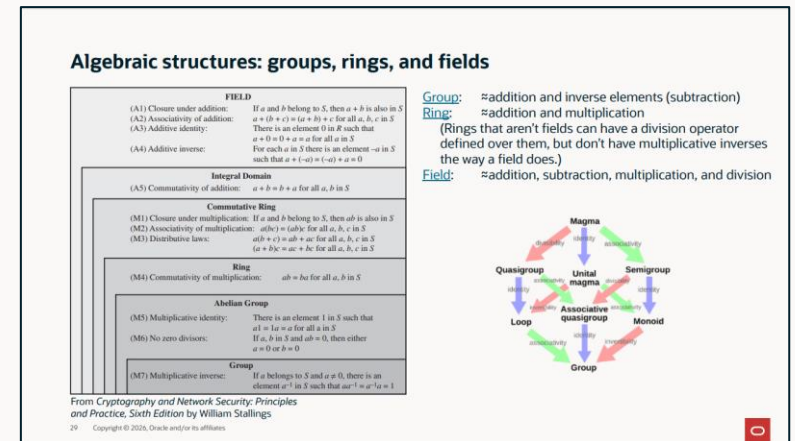
See <https://github.com/openjdk/valhalla/pull/1957>

Interfaces to declare various algebraic properties hold (placeholder names to minimize conflicts with existing types):

- `public interface MathGroup<MG> extends Numerical<MG> {...}`
- `public interface MathAbelianGroup<MAG> extends MathGroup<MAG> {...}`
The add operation is required to be commutative.
- `public interface MathRing<MR> extends MathAbelianGroup<MR> {...}`
Multiplication is associative.
- `public interface MathField<MF> extends MathRing<MF> {...}`
Multiplication is associative and commutative.

Could be used to enable transformations by code generators and VMs.

- Possible future connection to [RelaxedMath](#), HT Emanuel Peter
- Possible future connection to [Project Babylon](#)



ORACLE